ROCm Navigator

Security & Reporting Layer Specifications

AMD Instinct™ Hackathon 2026

Enterprise Security, Compliance, and Report Generation Layer

Developer: Yashwant (Member 5)

Executive Summary

The Security & Reporting Layer of ROCm Navigator serves as the enterprise safety gate during CUDA to HIP migrations. It acts as a primary firewall, preventing security and credentials leaks, auditing code memory operations, and compiling compliance artifacts to assure runtime safety.

System Architecture Flow

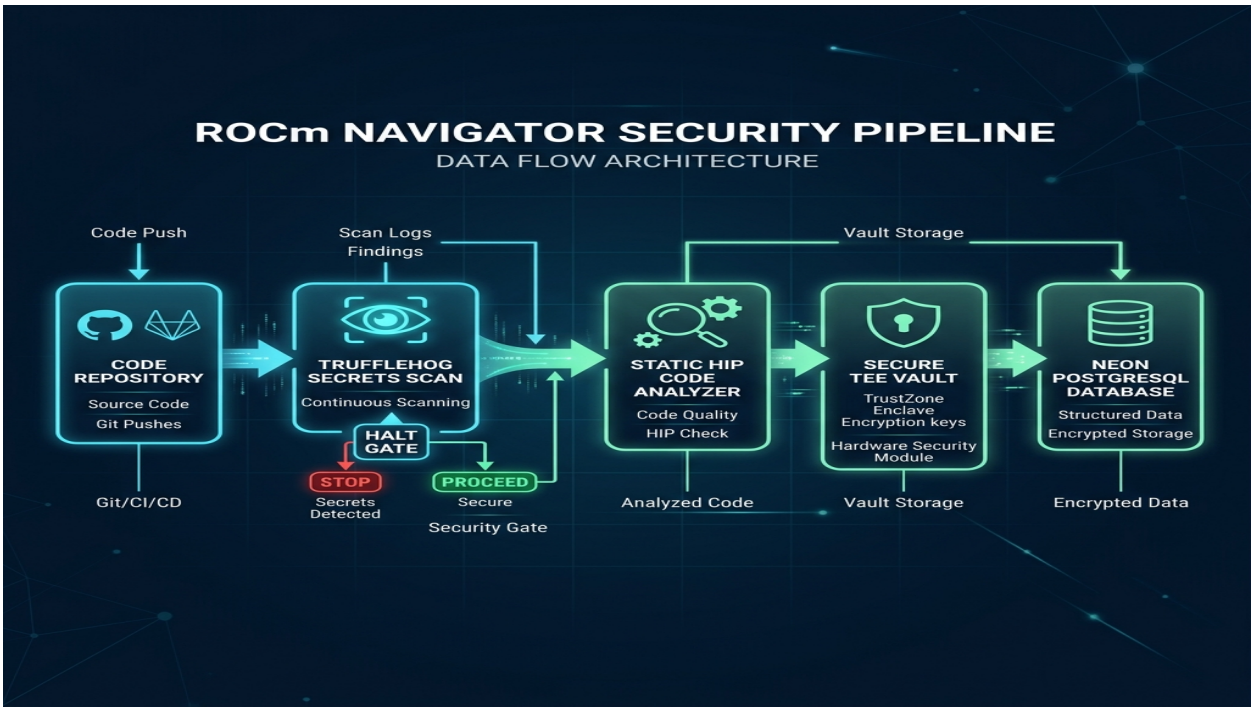

Figure 1: Pipeline authentication flow including Trufflehog validation, static analyzer gate, and Neon PostgreSQL database hooks.

Phase 1: Secret Scanning & Leak Prevention

Before any source file is submitted to downstream translation or synthesis models, the gateway executes an O(N) secret scan. Using the Trufflehog filesystem engine, the gateway audits files for verified secrets and credential leaks.

- **Halt Trigger:** If any verified credential is found, the gateway instantly raises an HTTP 403 Forbidden exception and stops the pipeline.
- **Scrubbing:** Active secrets are redacted inside user-facing logs and reports to prevent disclosure.

Phase 2: Static HIP Code Safety Analyzer

A rule-based static analyzer scans translated HIP code for dangerous memory access patterns. Key rules include:

- **HIP-MEM-001 / 002:** Validates that return values from 'hipMalloc' and 'hipMemcpy' are checked, preventing silent failures.
- **MEM-PTR-001:** Detects raw pointer arithmetic that could lead to out-of-bound (OOB) memory vulnerabilities.
- **HIP-OOB-001 / 002:** Monitors block and thread index manipulation routines to avoid memory segment boundary overruns.

Phase 3: Simulated TEE Secure Vault

To protect target repository credentials and configuration settings, the module implements a software-simulated AMD TEE (Trusted Execution Environment) Secure Vault. Key derivation is completed using PBKDF2 HMAC with a 16-byte cryptographically-secure random salt and 100,000 hashing rounds. Decrypted secrets are loaded directly into memory and never touch storage.

Phase 4: Neon PostgreSQL DB Integration

The module provides out-of-the-box support for both local development (SQLite) and production environments (Neon PostgreSQL). A runtime dialect translator dynamically converts query placeholders (`?` -> `%s`), handles auto-increments, and maps UPSERT queries between SQLite and PostgreSQL databases.

Table Name	Primary Key	Description
audit_logs	id (SERIAL)	System-wide event tracking and audit traces
security_scans	id (SERIAL)	Detailed scan safety stats and results
compiled_reports	report_id (VARCHAR)	Register and paths of generated reports

Security Dashboard & Metrics

Figure 2: Mockup dashboard showing active health scores, SBOM summary, and real-time security telemetry.

Verification & Test Results

The security module has been thoroughly validated through a comprehensive 72-item test suite. The test coverage confirms correct operation across the FastAPI gateway, PostgreSQL toggles, secrets scanner, SBOM generators, and report compiler.

Result Status: *72 / 72 tests passed successfully (100% Green). Database isolation is verified on Neon PostgreSQL.*